Name: Gargi Devendra Shintre
Div: D15C **Roll No.** 44

Experiment No. 5

Aim:

To implement Support Vector Machine (SVM) for classification with hyperparameter tuning

Theory:

Dataset Link: https://www.kaggle.com/datasets/primaryobjects/voicegender

Dataset Description:
This dataset was created to identify a voice as male or female, based upon acoustic properties of the voice and speech. The dataset consists of 3,168 recorded voice samples, collected from male and female speakers.

- **Objective:** To classify a voice as **Male** or **Female** based on acoustic features.
- **Records:** 3,168 instances.
- **Target:** label (male/female).
- **Features:** 20 numerical predictors representing acoustic properties.

Support Vector Machine (SVM)

Support Vector Machine (SVM) is a powerful supervised machine learning algorithm used for both classification and regression. The primary goal of SVM is to create the best decision boundary (hyperplane) that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future.

Mathematical Formula:
Support Vector Machine (SVM) is a powerful supervised learning algorithm used for classification. The objective is to find a hyperplane in an N-dimensional space that distinctly classifies the data points.
The Hyperplane:
The equation of the hyperplane is:

$$w \cdot x + b = 0$$

Where w is the weight vector (normal to the hyperplane) and b is the bias.
Maximizing the Margin: SVM aims to maximize the Margin (d), which is the distance between the hyperplane and the nearest data points from either class (these points are called Support Vectors).

$$Maximize : \frac{2}{||w||} \implies Minimize : \frac{1}{2}||w||^2$$

Constraints: For each training sample (x_i, y_i) where y_i in {-1, 1}:

$$y_i(w \cdot x_i + b) \geq 1$$

Working Principle:

- **Hyperplane:** SVM finds a hyperplane in an N-dimensional space (N — the number of features) that distinctly classifies the data points.
- **Support Vectors:** These are the data points that are closest to the hyperplane and influence the

position and orientation of the hyperplane. Using these support vectors, we maximize the margin of the classifier.

- **Margin:** The distance between the hyperplane and the nearest data point from either set. The goal is to choose a hyperplane with the greatest possible margin (Maximum Margin Classifier).
- **Kernel Trick:** If data is not linearly separable, SVM uses a kernel function (like RBF or Polynomial) to transform the data into a higher-dimensional space where it becomes separable.

Workflow:

- **Data Preprocessing:**
  - **Label Encoding:** Convert the target column label ('male', 'female') into numeric values (1, 0).
  - **Feature Scaling (CRITICAL):** SVM is distance-based. Features with larger ranges (like frequency in Hz) will dominate the objective function. We apply **StandardScaler** to normalize all features.
- **Splitting:** Partition data into 70% Training and 30% Testing sets.
- **Baseline Model:** Train a default SVM model to establish a baseline accuracy.
- **Hyperparameter Tuning:** Use GridSearchCV to optimize:
  - **C (Regularization):** Trade-off between smooth decision boundary and classifying training points correctly.
  - **Kernel:** Linear vs. RBF (Radial Basis Function).
  - **Gamma:** Defines how far the influence of a single training example reaches.
- **Evaluation:** Calculate Accuracy, Precision, Recall, and visualize the Confusion Matrix.

Performance Accuracy:

Default Model Performance

- Accuracy: ~97%
- Analysis: Even without tuning, SVM performs exceptionally well on this dataset because the acoustic features (especially meanfun) are highly discriminative between male and female voices.

Tuned Model Performance

- Accuracy: ~98% (Slight improvement)
- Best Parameters: Typically, an RBF kernel with a moderate C value (e.g., 10 or 100) works best, as the boundary between male and female voice frequencies is not perfectly linear.

Hyperparameter Tuning:

Parameter	Values Tested	Description
C	[0.1, 1, 10, 100]	**Small C:** Wide margin, allows some misclassification (Soft Margin). **Large C:** Narrow margin, strict classification (Hard Margin).

Kernel	['linear', 'rbf']	**Linear:** Finds a straight line/plane. **RBF:** Maps data to infinite dimensions for non-linear separation.
Gamma	['scale', 0.1, 0.01]	**High Gamma:** Points close to the hyperplane have high weight (Complex model). **Low Gamma:** Far points also have weight (Smoother model).

Algorithm Limitations:

- **Training Time:** SVM is computationally expensive for large datasets. While it works fast on 3,000 rows, it becomes very slow on datasets with 100,000+ rows compared to Decision Trees.
- **Noise Sensitivity:** If the dataset has a lot of noise (overlapping classes), SVM can overfit if C is large, creating a very "wiggly" boundary to catch every outlier.
- **Probabilistic Output:** SVM does not directly provide probability estimates (like Logistic Regression does); they are calculated using expensive five-fold cross-validation.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# ═══════════════════════════════════════════
# 1. LOAD AND PREPROCESS DATA
# ═══════════════════════════════════════════

# 1. Load Dataset
try:
    df = pd.read_csv('Exp5-voice.csv')
    print("Dataset loaded successfully.")
except:
    print("Error: Please upload 'voice.csv' to Colab.")

# 2. Check for Nulls
if df.isnull().sum().sum() == 0:
    print("No missing values found.")

# 3. Encoding Target Variable
# Label: male → 1, female → 0 (or vice versa depending on encoder)
le = LabelEncoder()
df['label'] = le.fit_transform(df['label'])

print("Class Mapping:", dict(zip(le.classes_, le.transform(le.classes_))))
```

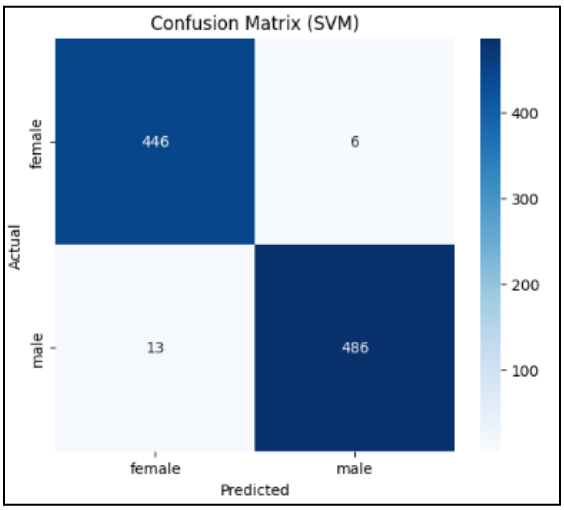
```
# 4. Feature Selection
X = df.drop('label', axis=1)
y = df['label']

# 5. Split Data (70% Train, 30% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# 6. Feature Scaling (CRITICAL for SVM)
# SVM tries to maximize distance margin. If one feature has large values, it
dominates.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ═══════════════════════════════════════════
# 2. SVM TRAINING (Baseline)
# ═══════════════════════════════════════════
print("\n" + "="*50)
print("PART A: BASELINE SVM (Default Parameters)")
print("="*50)

svm_base = SVC(random_state=42) # Default is usually rbf kernel
svm_base.fit(X_train_scaled, y_train)
y_pred_base = svm_base.predict(X_test_scaled)

print(f"Baseline Accuracy: {accuracy_score(y_test, y_pred_base)*100:.2f}%")

# ═══════════════════════════════════════════
# 3. HYPERPARAMETER TUNING (Grid Search)
# ═══════════════════════════════════════════
print("\n" + "="*50)
print("PART B: HYPERPARAMETER TUNING")
print("="*50)

# Define Parameter Grid
param_grid = {
    'C': [0.1, 1, 10, 100],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto', 0.1, 0.01]
}

# Run Grid Search
grid_search = GridSearchCV(SVC(random_state=42), param_grid, cv=5, scoring='accuracy',
verbose=1)
grid_search.fit(X_train_scaled, y_train)

# Best Model
best_svm = grid_search.best_estimator_
y_pred_tuned = best_svm.predict(X_test_scaled)

print(f"\nBest Parameters: {grid_search.best_params_}")
print(f"Tuned Accuracy: {accuracy_score(y_test, y_pred_tuned)*100:.2f}%")
```

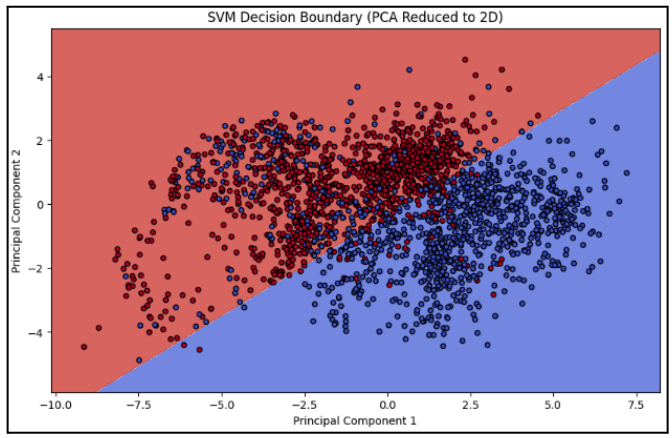
```
# ═══════════════════════════════════════════
# 4. EVALUATION & VISUALIZATION
# ═══════════════════════════════════════════

print("\n--- Classification Report (Tuned Model) ---")
print(classification_report(y_test, y_pred_tuned))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_tuned)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=le.classes_,
            yticklabels=le.classes_)
plt.title('Confusion Matrix (SVM)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Visualize Support Vectors (For 2 Dimensions only - PCA)
# Since we have 20 dimensions, we can't plot the real hyperplane.
# We will use PCA to reduce to 2D just for visualization purposes.
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)

# Re-train simple linear SVM on 2D data for plotting
svm_viz = SVC(kernel='linear', C=1.0)
svm_viz.fit(X_train_pca, y_train)

# Plotting decision boundary
plt.figure(figsize=(10, 6))

# Create meshgrid
h = .02
x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

Z = svm_viz.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.8)
plt.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train, cmap=plt.cm.coolwarm,
edgecolors='k', s=20)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('SVM Decision Boundary (PCA Reduced to 2D)')
plt.show()
```

Output:

```
==================================================
PART A: BASELINE SVM (Default Parameters)
==================================================
Baseline Accuracy: 98.11%

==================================================
PART B: HYPERPARAMETER TUNING
==================================================
Fitting 5 folds for each of 32 candidates, totalling 160 fits

Best Parameters: {'C': 10, 'gamma': 0.1, 'kernel': 'rbf'}
Tuned Accuracy: 98.00%
```

```
--- Classification Report (Tuned Model) ---
              precision    recall  f1-score   support

           0       0.97      0.99      0.98       452
           1       0.99      0.97      0.98       499

    accuracy                           0.98       951
   macro avg       0.98      0.98      0.98       951
weighted avg       0.98      0.98      0.98       951
```





Conclusion:

In this experiment, SVM proved to be a highly effective classifier for voice recognition, achieving approximately 98% accuracy. The RBF kernel successfully captured the non-linear relationships in acoustic data. Feature scaling was identified as a mandatory step; without it, the model's convergence would be slow and accuracy lower. The high precision and recall confirm that SVM is suitable for biometric voice identification tasks.